

ICS 483 Assignment 1

Justin Kugiyama

September 2026

Theory Questions

1.

Using $x = A \cos \phi$ and $y = A \sin \phi$, where $A = \sqrt{x^2 + y^2}$, we can rewrite the Hough equation as

$$\rho = x \cos \theta + y \sin \theta = A \cos \phi \cos \theta + A \sin \phi \sin \theta = A \cos(\theta - \phi).$$

Therefore, each image point produces a sinusoid in Hough space. Its amplitude is $A = \sqrt{x^2 + y^2}$, and its phase is $\phi = \tan^{-1}(y/x)$.

2.

We use (ρ, θ) instead of slope-intercept form because slope-intercept form cannot represent vertical lines and becomes unstable for very large slopes. Solving

$$\rho = x \cos \theta + y \sin \theta$$

for y gives

$$y = -\frac{\cos \theta}{\sin \theta}x + \frac{\rho}{\sin \theta}.$$

Which means the slope and intercept are

$$m = -\cot \theta, \quad c = \frac{\rho}{\sin \theta}.$$

3.

Since $x \in [1, W]$ and $y \in [1, H]$, the maximum absolute value of ρ occurs at the corner of the image that is farthest from the origin. Therefore, the maximum value is

$$|\rho|_{\max} = \sqrt{W^2 + H^2}.$$

Since a line represented by (ρ, θ) can be represented uniquely using angles from 0 to π , the range of θ is

$$0 \leq \theta < \pi.$$

4.

For this line, $m=1$ and $c=0$.

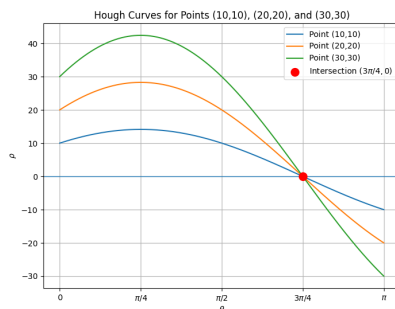


Figure 1: Hough Curves

Experiment Write-up

I tested the Hough line detector on the provided images using the full image-processing pipeline. Each image was first converted to grayscale and smoothed with a Gaussian filter using `myImageFilter`.

Next, `myEdgeFilter` used Sobel filters to find the horizontal and vertical image gradients. It calculated the edge magnitude and applied non-maximum suppression to make the edges thinner. An edge threshold of 0.03 was used to remove weak edge responses before applying the Hough transform.

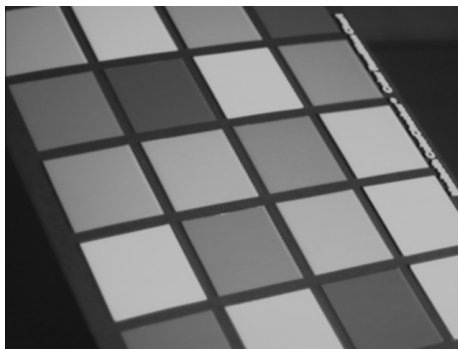


Figure 2: Filtered Image 1

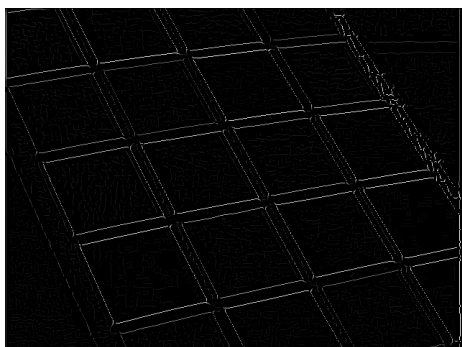


Figure 3: Edges Image 1

The Hough transform converted the detected edge points into possible lines using

$$\rho = x \cos(\theta) + y \sin(\theta).$$

The resulting Hough accumulator showed which line parameters had the most supporting edge points.

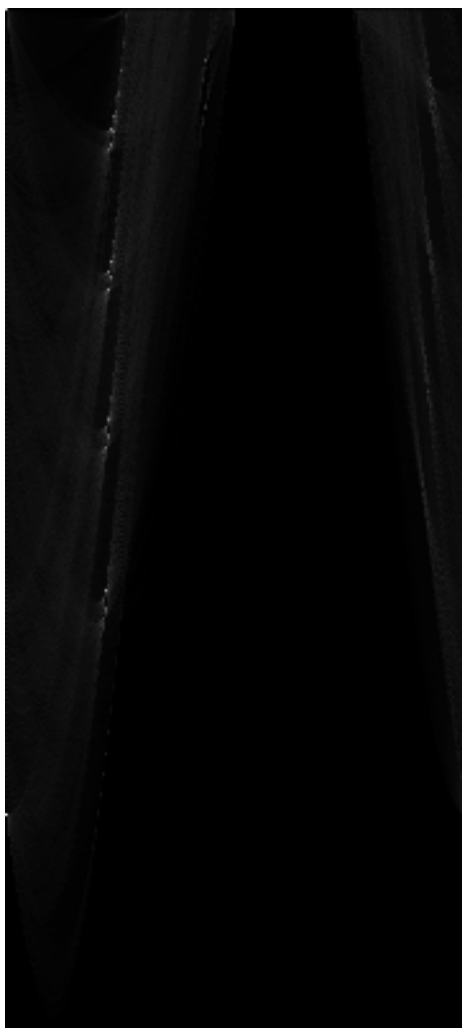


Figure 4: Hough Image 1

The strongest peaks in the accumulator were selected using `myHoughLines`. Non-maximum suppression was used to prevent several nearby peaks from being detected as separate copies of the same line. The lines found by my implementation are shown in red. The green line segments were produced by OpenCV's `HoughLinesP` function for comparison.

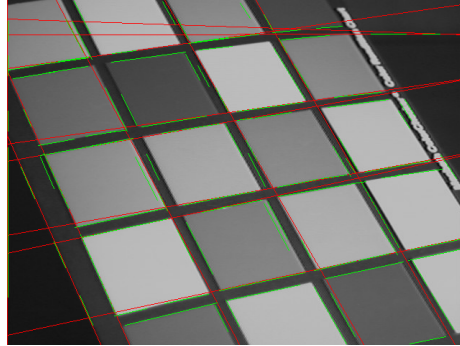


Figure 5: Hough Lines Image 1

Parameters and Results

The initial parameters were $\sigma = 2$, an edge threshold of 0.03, $\rho = 2$, $\theta = \pi/90$, and `nLines`=15.

The parameter σ controls the amount of smoothing. When this was a larger value more noise and small details were removed, but it also removed important edges. A smaller value kept more detail but may have created extra unwanted edges. Similarly, a lower edge threshold detected more edges but included noise, while a higher threshold produces a cleaner result but can remove weak lines.

The ρ and θ resolutions control the accuracy of the Hough accumulator. Smaller values give more precise line estimates but require more computation. The value of `nLines` controls how many detected lines are returned. Increasing it can find more lines, but it may also add weak or incorrect detections.

One set of parameters did not work equally well for every image. Images with clear, strong edges worked well with the initial settings. Images with more noise or small details required changes to the smoothing value and edge threshold. The best value of `nLines` also depended on how many important lines were present in each image.

The most difficult part was selecting the strongest Hough peaks. A single real line can produce several nearby peaks in the accumulator. Non-maximum suppression helped reduce these duplicate detections. Suppression during edge detection also improved the results by making the edges thinner before the Hough transform was applied.

Overall, I felt the pipeline worked successfully and demonstrated the full process of smoothing, edge detection, Hough transformation, and line detection. The final results depended heavily on the quality of the edge image and the parameters chosen for each image.

AI Usage

AI was mainly used to help plan the implementation and debug errors. I would start off by making an outline for what needs to be completed for the four functions to work, and then used generative AI to fill in the functions with python syntax.

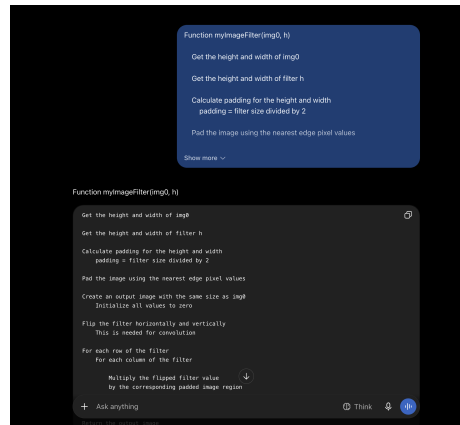


Figure 6: AI Prompt

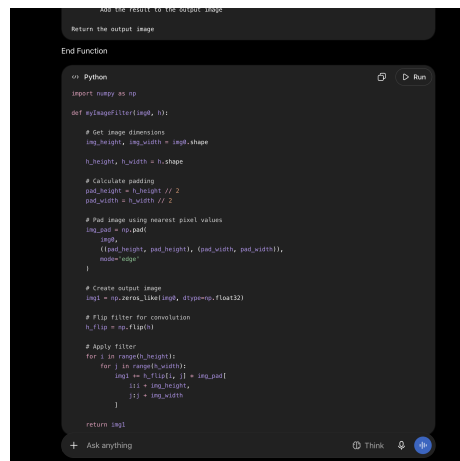


Figure 7: AI Prompt 2

After doing this, I would problem shoot and test the functions on the provided images. I also had generated some helper test functions to see the individual functions themselves. I did not run into any issues with the first two functions, filter and line detection, but for the hough problems, I thought the function

was not properly working. I checked the first two images from the data and the results looked very similar, to which I thought it was just using the same image or the parameters were getting interfered with from another function. I used GitHub Copilot to check the functions to see if there was any interference or conflicts between them.

```
Analyzed array shapes and updated  
myHoughLines.py  
  
The problem is in myHoughLines.py  
— it reshapes rhos / thetas into  
column vectors ( shape (nLines, 1) ),  
so thetaScale[thetas[k]] in  
houghScript.py returns a 1-element  
array instead of a scalar. That  
produces the deprecation warnings  
and ultimately breaks cv2.line ,  
which needs plain Python ints/floats  
for its point tuples.  
  
Reviewed and updated myHoughLines.py  
  
+0 -4
```

Figure 8: GitHub Copilot

This helped to find the problem and was able to finish testing with the other images to see how the results looked. Lastly, I used AI to help with formatting for LaTeX